

Data Structures Assignment

Pseudocode for Every Diagram — Array Queue (Part 1) & Linked List (Task A2)

PART 1 — QUEUE (ARRAY IMPLEMENTATION)

A queue of 6 slots (index 0 to 5) is used to serve students: **Maria (221045678)**, **Thomas (222034512)**, **Ndapewa (223041876)**, **Simon (221067341)**, **Peter (225005005)**, **Selma (226006006)**. The queue is a straight (non-circular) array: FRONT marks the index of the first occupied slot and REAR marks the index of the last occupied slot. When an element is removed, every remaining element shifts one place to the left.

Queue structure and setup (used by every diagram below)

```
CONSTANT MAX_SIZE = 6
ARRAY queue[MAX_SIZE]           // each slot stores (ID, Name)
FRONT = -1                       // index of the front element, -1 = empty
REAR  = -1                       // index of the rear (last) element
```

Diagram 1 — Building the queue (ENQUEUE) — full queue, FRONT=Maria, REAR=Selma

Enqueue adds a new element at the REAR end of the array.

```
BEGIN Enqueue(queue, value)
  IF REAR == MAX_SIZE - 1 THEN
    PRINT "Queue is Full - Overflow"
    RETURN
  END IF
  IF FRONT == -1 THEN           // inserting the very first element
    FRONT = 0
  END IF
  REAR = REAR + 1
  queue[REAR] = value
END Enqueue
```

Trace — matches Diagram 1: Enqueue is called six times, in this order:

Call	Value enqueued	FRONT	REAR	Queue contents (index 0..5)
1	Maria 221045678	0	0	Maria
2	Thomas 222034512	0	1	Maria, Thomas
3	Ndapewa 223041876	0	2	Maria, Thomas, Ndapewa
4	Simon 221067341	0	3	Maria, Thomas, Ndapewa, Simon
5	Peter 225005005	0	4	Maria, Thomas, Ndapewa, Simon, Peter
6	Selma 226006006	0	5	Maria, Thomas, Ndapewa, Simon, Peter, Selma

After call 6 the array is full (REAR = MAX_SIZE-1 = 5) — exactly the state drawn in Diagram 1.

Diagram 2 — Removing Maria, Thomas, Ndapewa (DEQUEUE with left-shift)

Because this is a straight array (not circular), Dequeue removes the FRONT element and then shifts every remaining element one position to the left so the queue always starts back at index 0.

```
BEGIN Dequeue(queue)
  IF FRONT == -1 OR FRONT > REAR THEN
    PRINT "Queue is Empty - Underflow"
    RETURN NULL
  END IF
  removedValue = queue[FRONT]
```

```

FOR i = FRONT TO REAR - 1 DO
    queue[i] = queue[i + 1]    // shift left
END FOR
queue[REAR] = EMPTY
REAR = REAR - 1
IF REAR < FRONT THEN          // queue became empty
    FRONT = -1
    REAR = -1
END IF
RETURN removedValue
END Dequeue

```

Trace — matches Diagram 2: Dequeue() is called three times on the full queue from Diagram 1:

Call	Value returned	FRONT	REAR	Queue contents after shift
1	Maria 221045678	0	4	Thomas, Ndapewa, Simon, Peter, Selma
2	Thomas 222034512	0	3	Ndapewa, Simon, Peter, Selma
3	Ndapewa 223041876	0	2	Simon, Peter, Selma

Result: Simon, Peter, Selma now sit at indices 0, 1, 2 with FRONT = 0, REAR = 2 — exactly Diagram 2.

Diagram 3 — "RETURN ONLY": the value handed back by Dequeue

This diagram isolates the RETURN step of the same Dequeue algorithm above: before the shifting happens, the FRONT element's data is copied out and returned to the caller. Applying Dequeue() once more to the state left by Diagram 2:

```

removedValue = queue[FRONT]    // queue[0] = Simon 221067341
RETURN removedValue            // --> SIMON 221067341 (returned to caller)
// the shifting/decrement lines then run as in Diagram 2's Dequeue,
// leaving Peter and Selma at indices 0 and 1

```

This is what the arrow labelled "RETURN ONLY" in Diagram 3 depicts: SIMON 221067341 is the value produced by the function call.

Diagram 4 — FRONT / REAR / EMPTY layout (isEmpty and isFull)

This diagram labels the parts of the queue: occupied slots (Simon, Peter, Selma) next to unused/EMPTY slots, with FRONT and REAR pointers marked. It illustrates the checks used to detect an empty or full queue.

```

BEGIN isEmpty(queue)
    IF FRONT == -1 THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END isEmpty

BEGIN isFull(queue)
    IF REAR == MAX_SIZE - 1 THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END isFull

```

At the state in Diagram 4 (Simon, Peter, Selma at 0..2, slots 3..5 EMPTY): isEmpty() = FALSE, isFull() = FALSE.

Additional queue operations (for completeness — not separately diagrammed)

Peek / Front — view the front value without removing it

```
BEGIN Peek(queue)
  IF isEmpty(queue) == TRUE THEN
    PRINT "Queue is Empty"
    RETURN NULL
  END IF
  RETURN queue[FRONT]
END Peek
```

Display / Traverse — print every element from FRONT to REAR

```
BEGIN Display(queue)
  IF isEmpty(queue) == TRUE THEN
    PRINT "Queue is Empty"
    RETURN
  END IF
  FOR i = FRONT TO REAR DO
    PRINT queue[i].ID, queue[i].Name
  END FOR
END Display
```

TASK A2 — SINGLY LINKED LIST

Each node stores a student's ID, Name, Task type and Time, plus a NEXT pointer (drawn in the diagrams as an address number, e.g. 10, 20, 30, or NULL for the last node). HEAD points to the first node, TAIL points to the last node.

Node structure and pointers (used by every diagram below)

```
STRUCTURE Node
  ID
  Name
  TaskType
  Time
  Next          // address of the next node, or NULL
END STRUCTURE
```

HEAD = address of the first node
TAIL = address of the last node

Diagram 1 — Starting list (before inserting Ndapewa)

HEAD points to Maria. The list is: Maria(10) -> Thomas(20) -> NULL, with TAIL = Thomas.

Address	ID	Name	Task	Time	Next
10	221045678	Maria	Registration	12 mins	20
20	222034512	Thomas	Student Card	5 mins	NULL

Diagram 2 — Insert Ndapewa at the beginning (INSERT AT HEAD)

```
BEGIN InsertAtBeginning(HEAD, TAIL, newNode)
  newNode.Next = HEAD
  HEAD = newNode
  IF TAIL == NULL THEN          // list was empty
    TAIL = newNode
  END IF
END InsertAtBeginning
```

Trace — matches Diagram 2: newNode = Ndapewa (address 5).

Step	Action	Result
1	newNode.Next = HEAD	Ndapewa(5).Next = 10 (Maria)
2	HEAD = newNode	HEAD = 5 (Ndapewa)

Final list: Ndapewa(5) -> Maria(10) -> Thomas(20) -> NULL, HEAD = 5, TAIL = Thomas — exactly Diagram 2's "After insert" state.

Diagram 3 — Insert Simon at the end (INSERT AT TAIL)

```
BEGIN InsertAtEnd(HEAD, TAIL, newNode)
  newNode.Next = NULL
  IF HEAD == NULL THEN          // list was empty
    HEAD = newNode
    TAIL = newNode
    RETURN
  END IF
  TAIL.Next = newNode
  TAIL = newNode
END InsertAtEnd
```

Trace — matches Diagram 3: newNode = Simon (address 30), starting from Ndapewa(5)->Maria(10)->Thomas(20)->NULL.

Step	Action	Result
1	TAIL.Next = newNode	Thomas(20).Next = 30 (Simon)
2	TAIL = newNode	TAIL = 30 (Simon)

Final list: Ndapewa(5) -> Maria(10) -> Thomas(20) -> Simon(30) -> NULL, TAIL = Simon — exactly Diagram 3's "After inserting Simon at the end".

Diagram 4 — Insert Peter after the 3rd position (INSERT AFTER N)

```

BEGIN InsertAfterPosition(HEAD, TAIL, position, newNode)
  IF position == 0 THEN
    CALL InsertAtBeginning(HEAD, TAIL, newNode)
    RETURN
  END IF
  current = HEAD
  count = 1
  WHILE current != NULL AND count < position DO
    current = current.Next
    count = count + 1
  END WHILE
  IF current == NULL THEN
    PRINT "Position out of range"
    RETURN
  END IF
  newNode.Next = current.Next
  current.Next = newNode
  IF newNode.Next == NULL THEN      // inserted after the old tail
    TAIL = newNode
  END IF
END InsertAfterPosition

```

Trace — matches Diagram 4: position = 3 (1=Ndapewa, 2=Maria, 3=Thomas), newNode = Peter (address 25).

Step	Action	Result
1	current walks to position 3	current = 20 (Thomas)
2	newNode.Next = current.Next	Peter(25).Next = 30 (Simon)
3	current.Next = newNode	Thomas(20).Next = 25 (Peter)

Final list: Ndapewa(5) -> Maria(10) -> Thomas(20) -> Peter(25) -> Simon(30) -> NULL — exactly Diagram 4.

Diagram 5 — Delete after the 2nd position: removes Thomas (DELETE MIDDLE NODE)

```

BEGIN DeleteAfterPosition(HEAD, TAIL, position)
  current = HEAD
  count = 1
  WHILE current != NULL AND count < position DO
    current = current.Next
    count = count + 1
  END WHILE
  IF current == NULL OR current.Next == NULL THEN
    PRINT "Position out of range / nothing to delete"
    RETURN
  END IF
  nodeToDelete = current.Next
  current.Next = nodeToDelete.Next      // bridge over the deleted node
  IF nodeToDelete == TAIL THEN
    TAIL = current
  END IF

```

```

    FREE nodeToDelete
END DeleteAfterPosition

```

Trace — matches Diagram 5 ("Deletion after second position" / "Deletion of the middle node"): position = 2 (1=Ndapewa, 2=Maria).

Step	Action	Result
1	current walks to position 2	current = 10 (Maria)
2	nodeToDelete = current.Next	nodeToDelete = 20 (Thomas)
3	current.Next = nodeToDelete.Next	current.Next = 25 (Peter)
4	FREE nodeToDelete	Thomas node removed

Final list (matches the "final look" diagram): Ndapewa(5) -> Maria(10) -> Peter(25) -> Simon(30) -> NULL, HEAD = 5, TAIL = Simon.

Diagram 6 — Final list / Display (TRAVERSAL)

Requested addition: traversal walks the list from HEAD to NULL, visiting every node exactly once.

```

BEGIN Traverse(HEAD)
    current = HEAD
    count = 0
    IF current == NULL THEN
        PRINT "List is Empty"
        RETURN
    END IF
    WHILE current != NULL DO
        PRINT current.ID, current.Name, current.TaskType, current.Time
        count = count + 1
        current = current.Next
    END WHILE
    PRINT "Total nodes visited:", count
END Traverse

```

Trace on the final list: prints Ndapewa, Maria, Peter, Simon in that order; Total nodes visited = 4 — this is the "final look" state.

Additional operation — SEARCH by ID

Requested addition: search walks the list from HEAD until the target ID is found or the list ends.

```

BEGIN Search(HEAD, targetID)
    current = HEAD
    position = 1
    WHILE current != NULL DO
        IF current.ID == targetID THEN
            PRINT "Found:", current.Name, "at position", position
            RETURN current
        END IF
        current = current.Next
        position = position + 1
    END WHILE
    PRINT "Not Found"
    RETURN NULL
END Search

```

Trace on the final list: Search(HEAD, 225005005) walks Ndapewa(1) -> Maria(2) -> Peter(3) and returns Peter, found at position 3.

Additional operations (for completeness)

isEmpty — check whether the list has any nodes

```

BEGIN isEmpty(HEAD)
    IF HEAD == NULL THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END isEmpty

```

Delete at Beginning — remove the HEAD node

```

BEGIN DeleteAtBeginning(HEAD, TAIL)
    IF HEAD == NULL THEN
        PRINT "List is Empty"
        RETURN
    END IF
    nodeToDelete = HEAD
    HEAD = HEAD.Next
    IF HEAD == NULL THEN          // list became empty
        TAIL = NULL
    END IF
    FREE nodeToDelete
END DeleteAtBeginning

```

Delete at End — remove the TAIL node

```

BEGIN DeleteAtEnd(HEAD, TAIL)
    IF HEAD == NULL THEN
        PRINT "List is Empty"
        RETURN
    END IF
    IF HEAD == TAIL THEN          // only one node
        FREE HEAD
        HEAD = NULL
        TAIL = NULL
        RETURN
    END IF
    current = HEAD
    WHILE current.Next != TAIL DO
        current = current.Next
    END WHILE
    FREE TAIL
    current.Next = NULL
    TAIL = current
END DeleteAtEnd

```